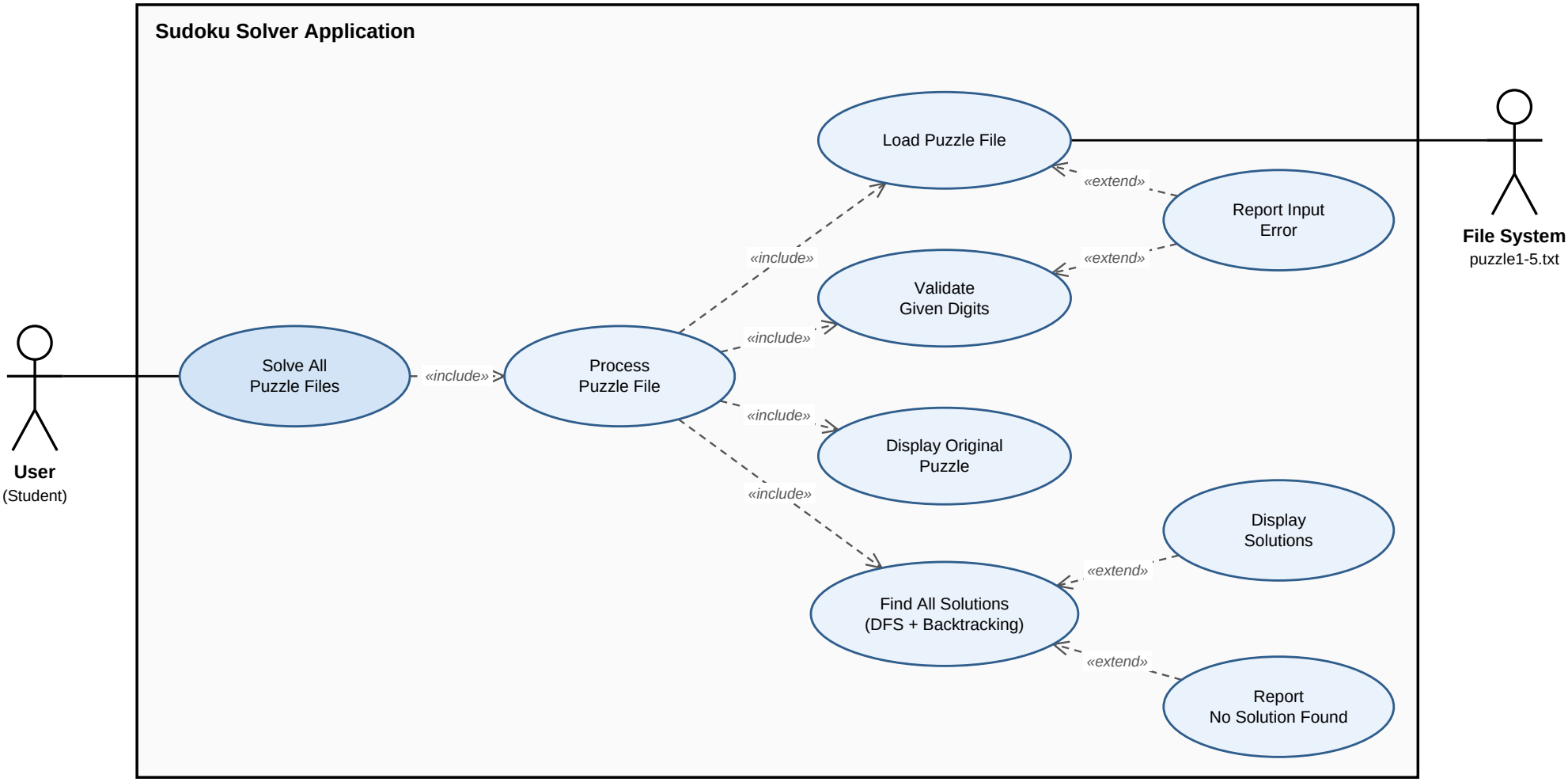
Assignment 4 - Sudoku Solver: UML Use Case Diagram

EECS 348 - Software Engineering I



Legend

- Association: actor participates in the use case
- «include»: base use case always uses the included use case
- «extend»: conditional behavior; arrow points to the base use case

Notes

Report Input Error: missing file, invalid character, not exactly 81 cells, or conflicting givens.

Display Solutions: runs once for every complete valid board found.

Report No Solution Found: runs when no completion of the puzzle exists.

Diagram and use case description drafted with Claude (Anthropic).

Assignment 4 - Sudoku Solver: Use Case Description

EECS 348 - Software Engineering I

a. Identified Use Case

Use case ID	UC-1
Use case name	Solve All Puzzle Files
Primary actor	User (the student running Assignment4.exe)
Secondary actor	File System (supplies puzzle1.txt through puzzle5.txt)
Scope	Sudoku Solver Application
Goal	In a single run, see the original puzzle and every valid solution (or a clear failure message) for each of the five puzzle files.

Supporting use cases shown in the diagram:

Use case	Relationship	Purpose
Process Puzzle File	Included by UC-1	Handles one puzzle file from loading to final result; repeated for each of the five files.
Load Puzzle File	Included by Process Puzzle File	Reads exactly 81 cells (digits 1-9 or "_") from the file, ignoring whitespace.
Validate Given Digits	Included by Process Puzzle File	Confirms that no given digit repeats in its row, column, or 3x3 block.
Display Original Puzzle	Included by Process Puzzle File	Prints the puzzle as read, using "_" for blank cells.
Find All Solutions	Included by Process Puzzle File	Recursive depth-first search with backtracking that enumerates every valid completion.
Display Solutions	Extends Find All Solutions	Prints each complete valid board as soon as it is found.
Report No Solution Found	Extends Find All Solutions	Prints "No solution found" when the search finds no valid completion.
Report Input Error	Extends Load Puzzle File and Validate Given Digits	Prints an error message when a file is missing, malformed, or has conflicting givens.

b. Brief Description

The User runs the Sudoku Solver, which processes puzzle1.txt through puzzle5.txt one after another. For each file the system reads the puzzle, prints it, checks the given digits, and uses a recursive depth-first search with backtracking to find every valid solution. Each solution is printed; if none exists, the system prints "No solution found". A file that is missing, malformed, or contradictory is reported and skipped so the remaining puzzles are still solved.

c. Step-by-Step Flow of Events

Main flow

1. User runs Assignment4.exe from the folder that contains the five puzzle files.
2. System selects the next puzzle file (starting with puzzle1.txt) and prints "Puzzle file: <name>".
3. System reads the 81 cells from the file. *(Load Puzzle File)*
4. System prints "Original puzzle:" followed by the 9x9 grid. *(Display Original Puzzle)*
5. System checks that the given digits do not conflict with one another. *(Validate Given Digits)*
6. System searches for solutions: it picks the blank cell with the fewest legal candidates, tries each candidate in turn, recurses, and backtracks when a cell has no candidates. *(Find All Solutions)*
7. Each time every cell is filled, System prints "Solution:" and the completed grid. *(Display Solutions)*
8. When every possibility has been tried, System moves to the next file and repeats steps 2 to 7.
9. After the fifth file, System ends the run with exit status 0.

Alternate flows

3a. The file cannot be opened: System prints an error and skips the file. *(Report Input Error)*

3b. The file has an invalid character, more or fewer than 81 cells, or a read failure: System prints an error and skips the file. *(Report Input Error)*

5a. Two given digits conflict: System prints an error and "No solution found" and skips the search. *(Report Input Error)*

6a. The search finds no valid completion: System prints "No solution found". *(Report No Solution Found)*

7a. The puzzle has more than one solution: System prints every solution, each one once, in the order found.

After 3a, 3b, or 5a, the flow resumes at step 8 with the next file.

d. Detailed Description

Use case	UC-1 Solve All Puzzle Files
Actors	Primary: User. Secondary: File System.
Trigger	The User launches Assignment4.exe.
Preconditions	1. The program was compiled from Assignment4.cpp on the EECS cycle server. 2. puzzle1.txt through puzzle5.txt are in the folder the program is run from; the program looks for these exact names. 3. Each file is expected to hold 81 cells separated by whitespace: a digit 1-9 for a given cell or "_" for a blank.
Postconditions (success)	For each of the five files, the console shows the file name, the original puzzle, and every solution or "No solution found". The puzzle files are not modified, and the program exits with status 0.
Postconditions (input error)	A file with an input error is reported and skipped, the other files are still processed, and the program exits with status 1.

Main success scenario (detail)	**Load:** the system reads one non-whitespace character at a time. "_" becomes a blank (0) and 1-9 becomes a given. Exactly 81 characters must be read; anything else is an input error. The grid is stored only after the whole file has been read, so a bad file never leaves a half-loaded puzzle. **Validate:** each given digit is compared with the other cells in its row, its column, and its 3x3 block. Any repeat is a conflict. **Search:** at each step the system finds every blank cell and lists its legal candidates (digits not already in the cell's row, column, or block). If any blank has no candidates, that branch is abandoned. Otherwise it chooses the blank with the fewest candidates, places each candidate in turn, recurses, and clears the cell afterwards (backtracking). When no blank remains, the board is a solution and is printed. Because every branch is explored, all solutions are found.
Alternate flows (detail)	**A1 File cannot be opened (step 3):** error message printed; file skipped; error status recorded; continue with the next file. **A2 Malformed file (step 3):** invalid character, extra data beyond 81 cells, fewer than 81 cells, or read failure; handled as A1. **A3 Conflicting givens (step 5):** error message and "No solution found" printed; search skipped; error status recorded; continue with the next file. **A4 No solution (step 6):** "No solution found" printed. This is a valid result, not an input error, so the exit status is unaffected. **A5 Multiple solutions (step 7):** every solution is printed in the order found.
Includes	Process Puzzle File; Load Puzzle File; Validate Given Digits; Display Original Puzzle; Find All Solutions.
Extends	Display Solutions and Report No Solution Found (both extend Find All Solutions); Report Input Error (extends Load Puzzle File and Validate Given Digits).
Special requirements	Written in C++ as a single file using objects (SudokuPuzzle and SudokuApplication); must compile and run on the EECS cycle servers; the brute-force search should finish in a fraction of a second for the course puzzles.

Diagram and use case description drafted with Claude (Anthropic).